Technical Program & Product Case Study

Alejandra Badia
Technical Program Manager
End-to-End Product Delivery: Structured Content Platform

1. Project Overview

1.1. Project Title: PostBuilder – Structured Content Workflow & JSON Publishing Tool

1.2 Project Goal

Design and deliver a CMS-style content workflow that enables users to create, validate, preview, and export structured blog content as JSON. The project focuses on frontend-driven architecture, real-time feedback, and clean data modeling for modern publishing systems.

1.3 My Role

Owned product strategy, prioritization, and end-to-end delivery of a structured content platform MVP.

Product Leadership

- Defined MVP scope and made key product tradeoffs (frontend-first architecture, JSON output model)
- Prioritized backlog based on user value and dependency constraints
- Structured delivery into 2 sprints balancing UX and data integrity

Execution

- Delivered full-stack implementation to validate product decisions and ensure functionality

1.4 Methodology

Agile (Scrum-based workflow using Jira)

- Sprint planning
- Backlog prioritization
- Iterative feature delivery
- Continuous validation and refinement

1.5 System Relevance

Modern systems require structured, consistent outputs to enable reliable integration with APIs and downstream services.

PostBuilder demonstrates how enforcing data structure at the source improves interoperability, reduces errors, and ensures consistent data exchange across systems.

2. Backlog Management

2.1 The Problem

Systems lack a reliable way to generate structured, reusable content that can be directly consumed by APIs or downstream services without manual formatting.

2.2 Key Product Decision: Structured JSON Output

Treat the platform as a structured output system using JSON instead of traditional HTML to enable integration with API-driven and headless CMS workflows.

This enabled:

- Designing workflows for **structured JSON generation**
- Ensuring outputs are **system-compatible and reusable**

In addition, the project required organizing multiple features (UI, validation, backend processing) into a structured, prioritized workflow while ensuring clear development direction and scope control.

2.3 Key Tradeoffs

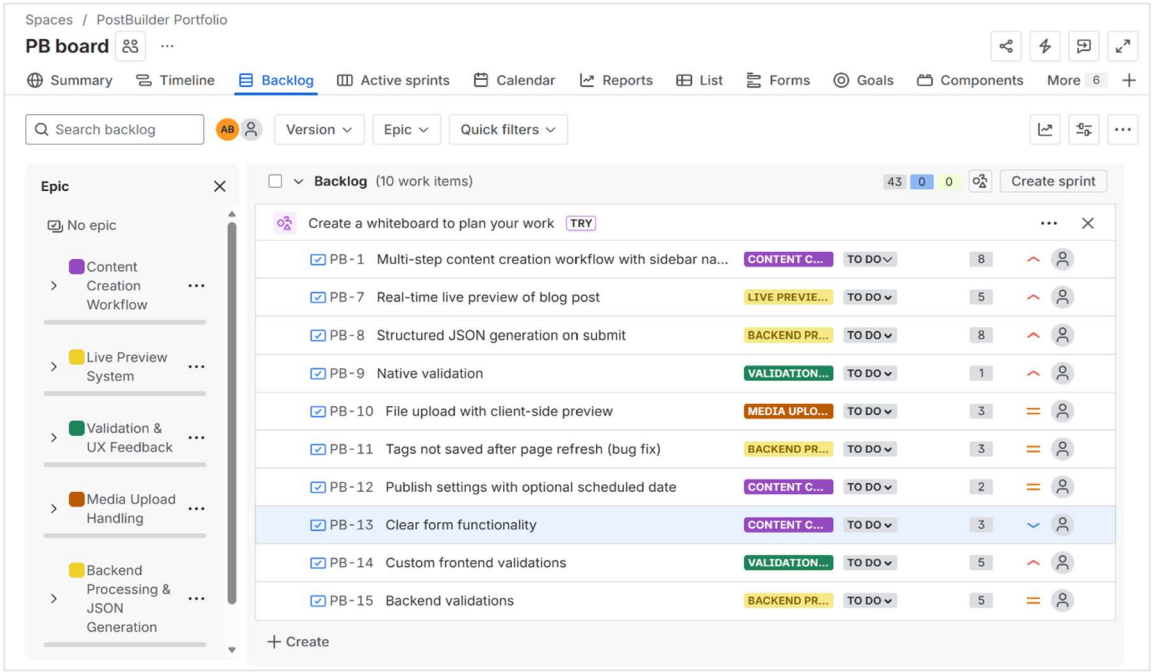
- Chose frontend-heavy architecture → faster iteration, less backend complexity
- Deferred advanced publishing features → prioritize core workflow
- Used synchronous validation first → reduce debugging complexity

2.4 Approach / Decision

- Created a structured backlog in Jira with:

  - Features (Stories)
  - Bug fixes
  - Acceptance criteria
  - Story points for estimation
- Grouped items under Epics (e.g., Content Workflow, Validation, Backend Processing)
- Prioritized high-impact features for early delivery (MVP-first approach)

2.5 Jira Backlog Screenshot



Screenshot 1 - Project Backlog: Structured backlog in Jira showing prioritized features, bug fixes, story points, and acceptance criteria. Items were organized under epics to align development with product goals and ensure clear scope definition.

2.6 Backlog Table

ID	Jira ID	Type	Summary	Epic	Story Points	Priority	Acceptance Criteria
F01	PB-1	Story	Multi-step content workflow	Content Workflow	8	High	Users can navigate sections; active state visible
F02	PB-7	Story	Real-time preview	Live Preview	5	High	All inputs reflected instantly in preview
F03	PB-8	Story	Structured JSON generation	Backend Processing	8	High	JSON includes all structured fields
F04	PB-9	Story	Native validation	Validation & UX Feedback	1	High	Required fields enforced; Character limits validated
F05	PB-10	Story	File upload preview	Media Upload	3	Medium	JPEG/PNG only; preview visible
B01	PB-11	Bug	Tags not saved after refresh	Backend Processing	3	Medium	Tags persist and submit correctly
F06	PB-12	Story	Publish settings	Content Workflow	2	Medium	Users can select publish options
F07	PB-13	Story	Clear form	Content Workflow	2	Low	Form resets completely
F08	PB-14	Story	Custom validation	Validation & UX Feedback	5	High	Custom invalid events trigger UX guidance
F09	PB-15	Story	Backend validations	Backend Processing	5	Medium	File size, type, mime validated, filename sanitized/validated; character limits validated; slug sanitized/validated

Table 1 - Backlog: Items were organized under epics to align development with product goals. Note: Issue IDs (PB-#) are automatically generated by Jira and are not sequential by item type.

3. Sprint 1 – Core Content Workflow

3.1 Sprint Goal

Deliver the core content creation experience including multi-step navigation, live preview, and validation.

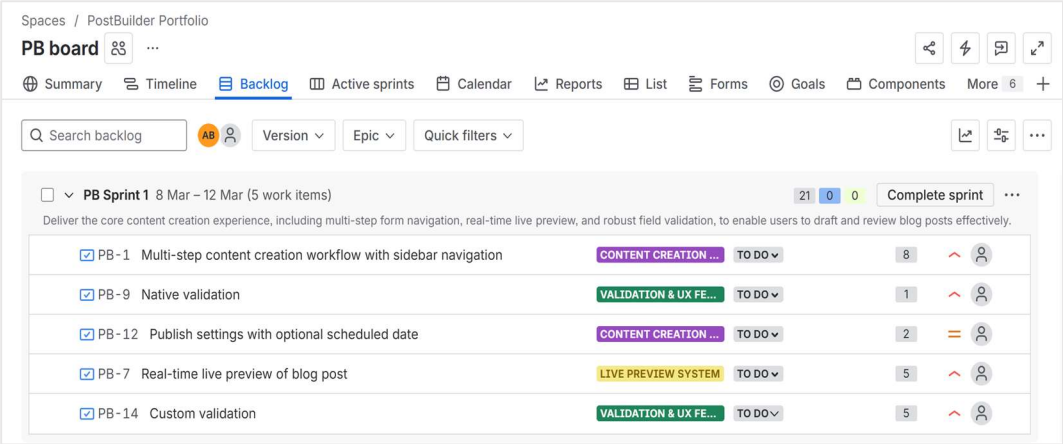
3.2 Initial Sprint State

3.2.1 **Problem:** Define and commit to a realistic scope for the first iteration while focusing on core user experience.

3.2.2 **Decision (Prioritization)**

- Selected high-priority features and highly dependent medium priority items
  - Navigation
  - Content creation / form
  - Live preview
  - Validation
- Deferred backend-heavy features to Sprint 2

3.2.3 **Sprint 1 To-Do Screenshot**



Screenshot 2 - Sprint 1 Initial State: Sprint 1 backlog committed with high-priority items and dependent medium priority items. Scope was intentionally limited to core user-facing functionality to ensure a stable MVP delivery.

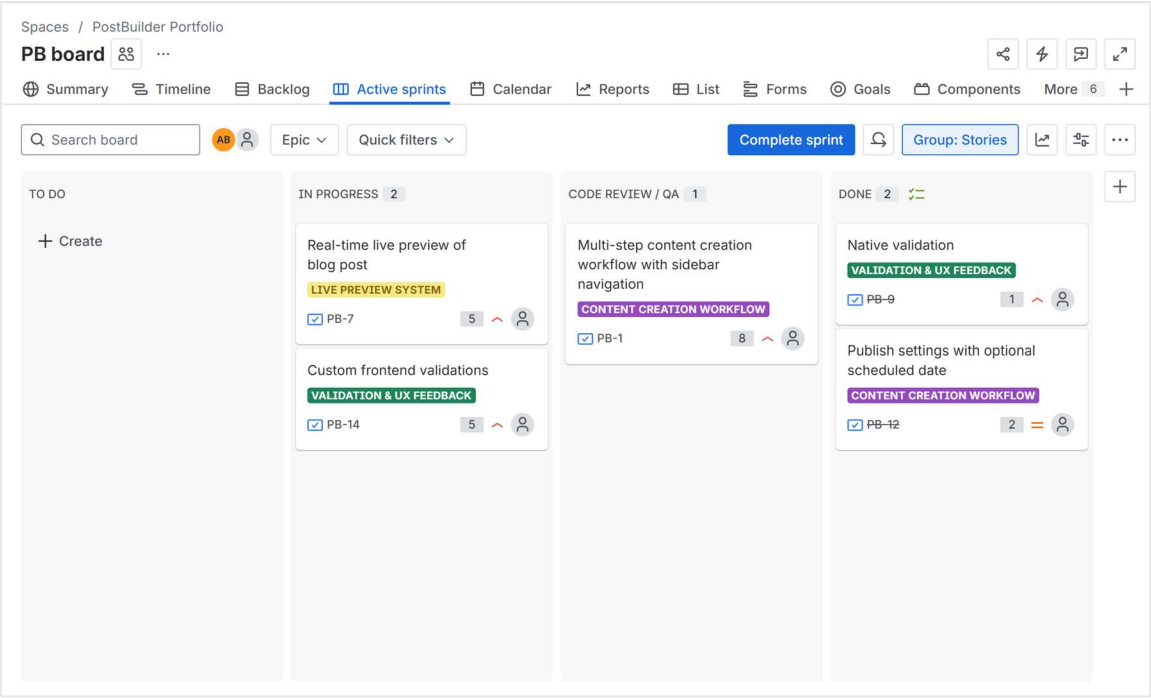
3.3 Work in Progress

3.3.1 **Problem:** Managing multiple interdependent frontend features (state, DOM updates, validation).

3.3.2 **Approach (Dependencies & Execution):**

- Live preview depended on state management
- Validation depended on form structure
- Navigation and content flow were structured to support publish settings and preview consistency
- Tasks were progressed in parallel but tested incrementally

3.3.3 Sprint 1 In-Progress Screenshot



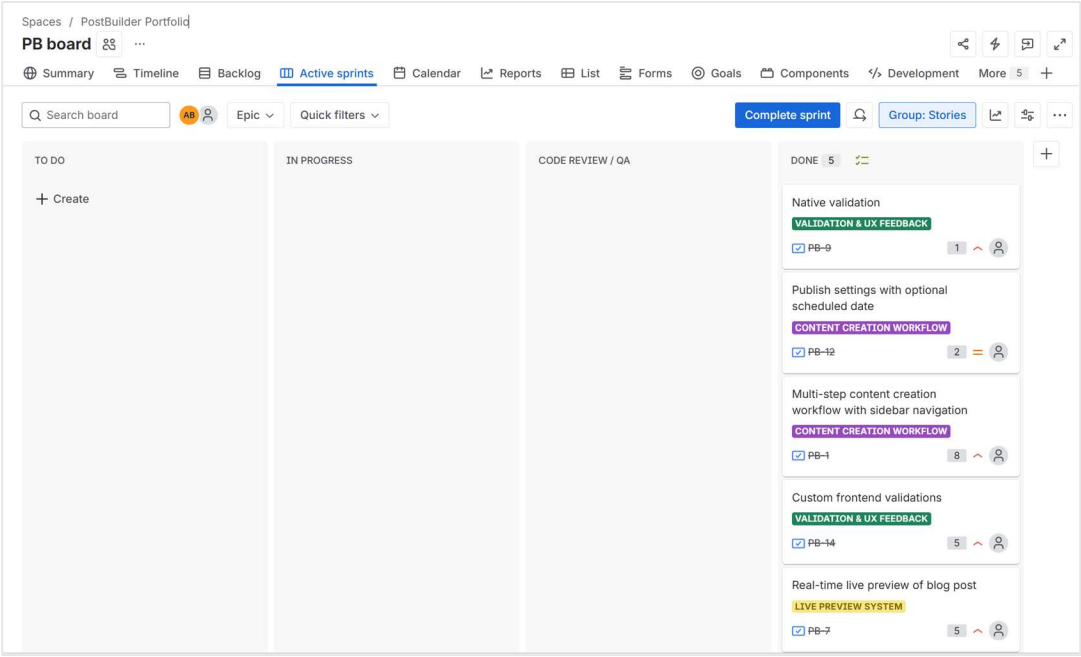
Screenshot 3 - Sprint 1 In-Progress State: Active Sprint 1 board showing items in progress. Dependencies between state management, preview rendering, and validation were managed to ensure consistent user experience.

3.4 Sprint Completion

3.4.1 Outcome

- Delivered fully functional:
  - Multi-step form
  - Live preview
  - Validation system
- Established stable frontend foundation:
  - Consistent UI/UX across multi-step workflow
  - Integrated native and custom validation handling
  - Centralized form and tag state to drive real-time preview updates and ensure consistent data across UI, autosave, and submission

3.4.2 Sprint 1 Completion Screenshot



Screenshot 4 - Sprint 1 Completion: Sprint 1 completed with all committed items delivered. Core content workflow established, enabling users to create and preview posts with real-time validation.

4. Sprint 2 – Data Handling & Output

4.1 Sprint Goal

This sprint focused on transforming user input into structured, system-ready JSON outputs, with JSON generation. File upload, form reset, backend validation, and resolving tag persistence issues, were also addressed in this sprint.

4.2 Initial Sprint State

4.2.1 Problem

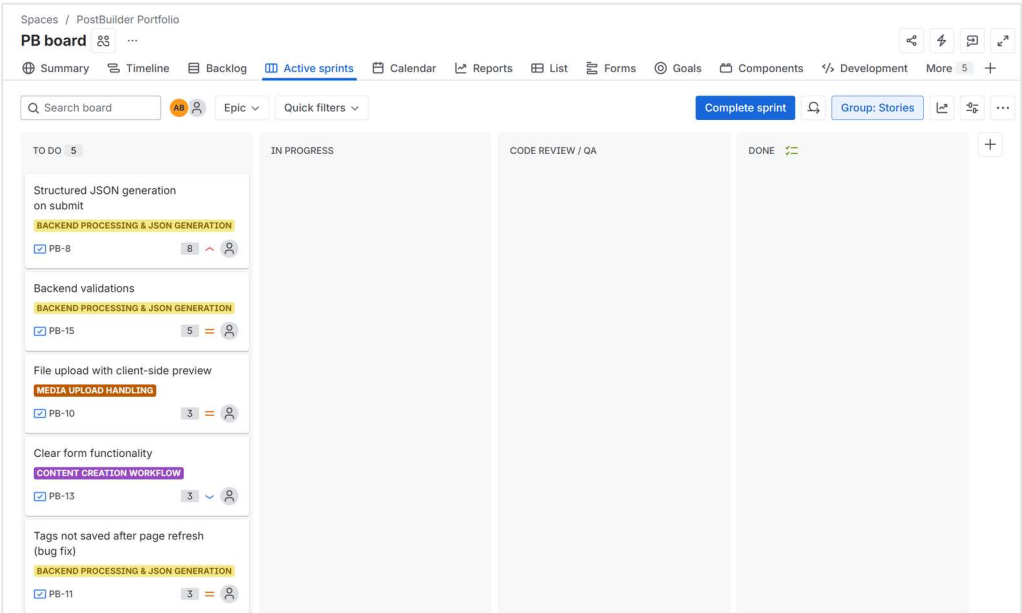
Extend functionality to include backend processing and data persistence without breaking existing UI flow.

4.2.2 Decision (Scope Management)

- Focused on:
  - Data output (JSON)
  - File handling
  - Bug resolution (tags issue)

- Maintained clear separation between frontend and backend responsibilities: the frontend handled user interaction and state management, while the backend ensured data validation, file handling, and JSON generation.

4.2.3 **Sprint 2 To-Do Screenshot**



Screenshot 5 - Sprint 2 Initial State: Sprint 2 backlog focused on backend integration and data handling. Scope was defined to extend functionality without introducing unnecessary complexity.

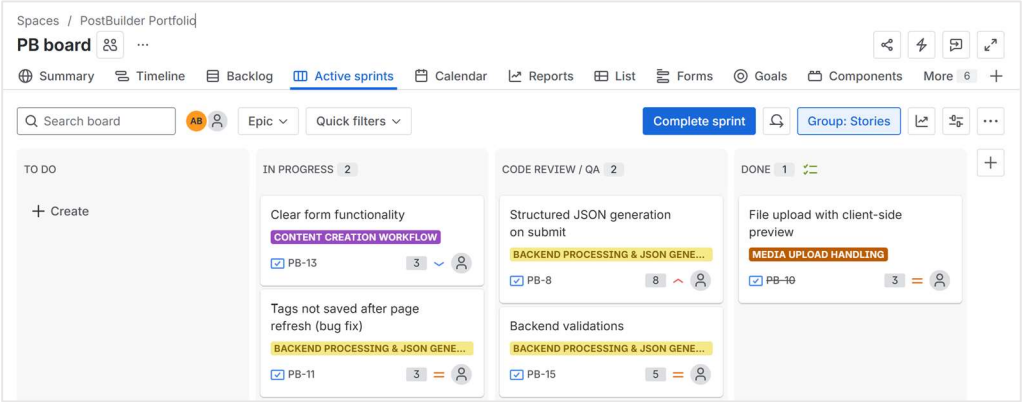
4.3 Work in Progress

4.3.1 **Problem:** Handling interaction between frontend state and backend submission (File preview, AJAX, PHP processing).

4.3.2 **Approach**

- Implemented backend JSON generation to transform structured form input into API-ready data, integrated with asynchronous frontend form submission (AJAX)
- Implemented backend validations, including file and character length validations
- Conducted iterative end-to-end testing of file upload and JSON output workflows, identifying and resolving inconsistencies between frontend state and backend processing

4.3.3 Sprint 2 In-Progress Screenshot



Screenshot 6 - Sprint 2 In-Progress State: Sprint 2 in progress, highlighting backend integration and bug resolution. Focus placed on ensuring data consistency between frontend inputs and server processing.

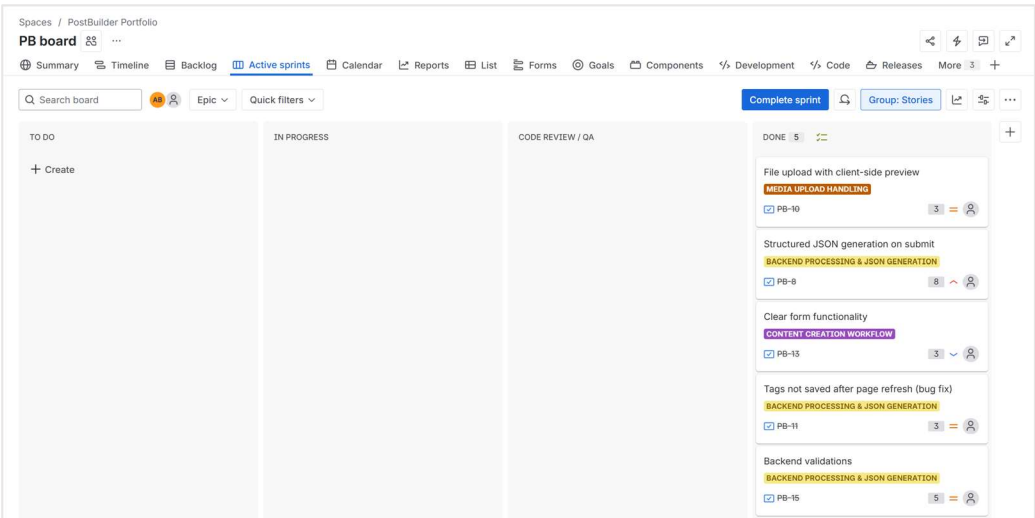
4.4 Sprint Completion

4.4.1 Outcome

Delivered:

- Structured JSON generation for API-ready content output
- File upload validation (type, size, and MIME restrictions)
- Clear form functionality with full UI and state reset
- Tag persistence fix ensuring consistent submission data

4.4.2 Sprint 2 Completion Screenshot



Screenshot 7 - Sprint 2 Completion: Sprint 2 completed with successful delivery of backend processing and data output features. The application now supports full content creation to structured JSON export workflow.

5. Metrics & Outcome

Product Outcomes:

- Enabled creation of API-ready structured content, reducing need for manual formatting before backend consumption
- Reduced invalid submission states through real-time validation, improving data reliability prior to processing
- Ensured consistent data integrity across frontend and backend through end-to-end validation workflows
- Enabled reliable data exchange between frontend and backend systems through enforced schema validation and end-to-end consistency checks

Delivery Outcomes:

- Delivered MVP in 2 structured sprints with controlled scope
- Prioritized core workflow first, preventing over-engineering and reducing scope creep
- Resolved critical data persistence issue (tags), improving reliability across user sessions

6. Project Management Insights

- **Prioritization:** Focused on user-facing features first (preview, validation)
- **Scope Management:** Deferred non-critical features to maintain delivery timeline
- **Dependencies:** Managed relationships between UI state, validation, and backend processing
- **Execution:** Used Jira to track progress, organize work, and maintain visibility

7. Supporting Artifacts

- Jira Backlog
- Sprint Boards (Sprint 1 & Sprint 2)

8. Security & Data Handling Considerations

- Sanitized user inputs (e.g., slug normalization, character limits)
- Prevented XSS by using safe DOM methods (e.g., textContent instead of innerHTML)
- Validated file uploads (type, MIME, size restrictions)
- Enforced backend validation alongside frontend checks
- Applied storage limits to prevent abuse

9. Product Thinking: What I'd Do Next

- Introduce content versioning to support iteration and rollback
- Add role-based workflows (editor, publisher)
- Integrate with headless CMS APIs for real-world publishing
- Add analytics on content usage and errors

10. Future Improvements

Potential enhancements may include:

- Database persistence
- CMS API integration
- Markdown editor support
- Autosave improvement
- Multi-user workflows and role-based permissions